

Report conversation

As a backend engineer reviewing this code, what kind of feedback what you have on the design of this function?

```

go
// TODO: Remove this function after the node
// Returns "200" if none of the pre-defined
func extractStatusFromResponse(response string) string {
    patterns := []*regexp.Regexp{
        regexp.MustCompile(`"code"\s*:\s*(\d{3})`),
        regexp.MustCompile(`(\d+)\s+Not Found`),
        regexp.MustCompile(`(\d+)\s+page not found`),
        regexp.MustCompile(`HTTP/\d\.\d\s+200`),
        regexp.MustCompile(`"statusCode"\s*:`),
    }

    for _, pattern := range patterns {
        matches := pattern.FindStringSubmatch(response)
        if len(matches) > 1 {
            return matches[1] // Return the status code
        }
    }
    return "200"
}

```

Here are some feedback points regarding the design and implementation of the `extractStatusFromResponse` function:

What's Good:

- **Specificity in Regex:** The function uses regular expressions tailored to different potential formats of status codes in responses, which is good for ensuring wide compatibility.
- **Early Return:** The function returns as soon as a match is found, which minimizes unnecessary processing once a status code is detected.

What Could Be Improved:

- **Performance Concerns:** Compiling regex patterns every time the function is called can be inefficient, especially under high load. Consider moving the compilation of regex patterns outside the function, possibly storing them as global variables if they do not change.
- **Hard-Coded Return Value:** Returning a hard-coded "200" when no patterns match can be misleading. This design assumes that the absence of an error code implies a successful operation, which might not always be the case. It's better to handle such scenarios more explicitly.
- **Error Handling:** The function lacks error handling. If the regular expressions are faulty or the input is not as expected, it silently defaults to "200". It might be beneficial to include logging or to return an error status to handle unexpected inputs or patterns better.
- **Lack of Flexibility:** The patterns are hard-coded, which makes the function less adaptable to changes in how responses are structured. Consider allowing the function to accept additional patterns as parameters, making it more flexible.
- **Code Documentation:** While there is a TODO comment, the function could benefit from more detailed comments explaining the purpose of each regex pattern and why these specific patterns were chosen.

Suggested Code with Improvements:

go

 Copy code

```
// Global patterns stored to avoid recompilation on each function call.
var patterns = []*regexp.Regexp{
    regexp.MustCompile(`"code"\s*:\s*(\d+)`), // Matches `"code": 4X
    regexp.MustCompile(`(\d+)\s+Not Found`), // Matches `404 Not Fou
    regexp.MustCompile(`(\d+)\s+page not found`), // Matches `404 page n
    regexp.MustCompile(`HTTP\//\d\.\d\s+(\d+)`), // Matches HTTP status
    regexp.MustCompile(`"statusCode"\s*:\s*(\d+)`), // Matches `"statusCode

}

// extractStatusFromResponse returns a status code extracted from the respo
// If no status code is found, it returns "unknown" to indicate unclear sta
func extractStatusFromResponse(response string) string {
    for _, pattern := range patterns {
        matches := pattern.FindStringSubmatch(response)
```

```
    if len(matches) > 1 {  
        return matches[1] // Return the first captured group, which sh  
    }  
}  
return "unknown" // Changed from "200" to "unknown" to avoid implying :
```

This revision includes moving the regex compilation to global scope for performance improvement and changes the default return value to "unknown" to better reflect uncertain outcomes.